L11A: Streams II

CS1101S: Programming Methodology

Martin Henz

28th October, 2026

Version 1.4 — Built: 2026-10-28 09:18:48 SGT

Announcements

- Mastery Check 2 begins now
- Topics
  - environment model
  - memoization
  - streams
- Teaching feedback is open; go to the feedback site
- Forum hours: one last session next Monday, 6:30pm, COM1 basement
- Consultation: drop an email to your teaching staff

Course Overview

- Unit 1—Functions (textbook Chapter 1)
  - Getting acquainted with the elements of programming, using **functional abstraction**
  - Learning to read programs, and using the **substitution model**
  - Example applications: Runes, curves
- Unit 2—Data (textbook Chapter 2)
  - Getting familiar with data: pairs, linked lists, trees, streams
  - Searching in linked lists and trees, sorting of linked lists
  - Example application: sound processing

Course Overview, continued

- Unit 3—State (parts of textbook Chapter 3)
  - Programming with stateful abstractions
  - Lists, loops, searching in and sorting of lists
  - Reading programs using the environment model
  - Example applications: robotics, video processing
- **Unit 4—Beyond (parts of textbook Chapters 3 and 4)**
  - Memoization, **Memoized Streams**
  - Metalinguistic abstraction

Outline

Review

Streams as Recursive Definitions

Memoization

Applications

Outline

Review

Streams as Recursive Definitions

Memoization

Applications

Idea of Streams

- Delayed linked lists
  - Our pairs contain a data item as `head` (as usual), but a function as `tail` that can be activated when needed
- Streams
  - A stream of a certain type is either `None`,
  - or a pair
    - whose `head` is of that type and
    - whose `tail` is a nullary function that returns a stream of that type
- Stream discipline
  - Like the linked-list discipline, now using streams

Examples — Finite Streams

```
# An empty stream
s1 = None
```



```
# A stream with element 1
s2 = pair(1, lambda: None)
```



```
# A stream with elements 1, 2, 3
s3 = pair(1,
          lambda: pair(2,
                       lambda: pair(3,
                                    lambda: None)))
```



Example — An Infinite Stream

```
def ones_stream():
    return pair(1, ones_stream)
ones = ones_stream()
def stream_tail(stream):
    return tail(stream)()
```



- head(ones) $\Rightarrow$ 1
- head(stream_tail(ones)) $\Rightarrow$ 1
- head(stream_tail(stream_tail(ones))) $\Rightarrow$ 1

Stream Processing

- Like linked lists, except
  - Wrap `tail` in a function
  - Use `stream_tail` instead of `tail`
- Stream support in Python §3
  - Pre-declared functions for stream processing
    - Examples: `stream_tail`, `stream_map`, `stream_filter`
  - Refer to Specification of Python §3 (`python_3.pdf`)

stream_map **and** stream_filter



```
def stream_map(f, s):
    if is_none(s):
        return None
    else:
        return pair(f(head(s)),
                    lambda: stream_map(f, stream_tail(s)))

def stream_filter(p, s):
    if is_none(s):
        return None
    elif p(head(s)):
        return pair(head(s),
                    lambda: stream_filter(p, stream_tail(s)))
    else:
        return stream_filter(p, stream_tail(s))
```

Streams of Integers

```
def integers_from(n):
    return pair(n, lambda: integers_from(n + 1))
integers = integers_from(1)
```



```
def eval_stream(s, n):
    return None if n == 0 else pair(head(s),
        eval_stream(stream_tail(s), n - 1))
```



```
eval_stream(integers, 5)
```



```
[1, [2, [3, [4, [5, None]]]]]
```

Why do we need streams?

- Streams allow us to represent infinite sequences!
- Streams allow us to use sequence manipulation (e.g. map, filter), while keeping the efficiency of incremental computation
- Example applications
- Video & Audio streaming
- Process and play A/V on-the-fly without entire file available
- Stream editing
- Process a file as a stream (e.g. find and replace text using AWK)

Outline

Review

Streams as Recursive Definitions

Memoization

Applications

Recall Stream of Ones

```
# original version
def ones_stream():
    return pair(1, ones_stream)
ones = ones_stream()
```



```
# alternative version
ones = pair(1, lambda: ones)
```



Defining a Repeating Sequence

- Wanted: Stream containing 1, 2, 3, 1, 2, 3, 1, 2, 3, ...

```
# A stream containing 1, 2, 3, 1, 2, 3, and so on
rep123 = pair(1, lambda: pair(2,
                              lambda: pair(3, lambda: rep123)))
eval_stream(rep123, 15)
```

```
[1, [2, [3, [1, [2, [3, [1, [2, [3, [1, [2, [3, [1, [2,
[3, None]]]]]]]]]]]]]]
```

Adding Two Streams

- Wanted: A function `add_streams` that takes two streams and returns a stream that contains the element-wise sums
- Example: Adding

  ```
  1 2 3 4 5 6 ...
  1 1 2 2 3 3 ...
  ```

- should return

  ```
  2 3 5 6 8 9 ...
  ```

Adding Streams

```
def add_streams(s1, s2):
    if is_none(s1):
        return s2
    elif is_none(s2):
        return s1
    else:
        return pair(head(s1) + head(s2),
                    lambda: add_streams(stream_tail(s1),
                                        stream_tail(s2)))
```



Fibonacci Numbers

```
fibs = pair(0, lambda: pair(1,
    lambda: add_streams(stream_tail(fibs), fibs)))
eval_stream(fibs, 10)
```



```
[0, [1, [1, [2, [3, [5, [8, [13, [21, [34, None]]]]]]]]]]
```

Fibonacci Numbers

```
         1  1  2  3  5   8  13 ... = stream_tail(fibs)
+        0  1  1  2  3   5   8 ... = fibs
-----------------------------------------------
  0  1  1  2  3  5  8  13  21 ... = fibs
```

```
fibs = pair(0, lambda: pair(1,
    lambda: add_streams(stream_tail(fibs), fibs)))
```



Integers Revisited

```
ones = pair(1, lambda: ones)
integers = pair(1, lambda: add_streams(ones, integers))
eval_stream(integers, 10)
```



```
[1, [2, [3, [4, [5, [6, [7, [8, [9, [10, None]]]]]]]]]]
```

Integers Revisited

```
       1  1  1  1  1  1 ... = ones
+      1  2  3  4  5  6 ... = integers
-------------------------------------
    1  2  3  4  5  6  7 ... = integers
```

```
ones = pair(1, lambda: ones)
integers = pair(1, lambda: add_streams(ones, integers))
```



Which Stream Is This?



```
def mul_streams(s1, s2):
    return pair(head(s1) * head(s2),
                lambda: mul_streams(stream_tail(s1),
                                    stream_tail(s2)))
mystery = pair(1, lambda: mul_streams(mystery, integers))
eval_stream(mystery, 8)
```

- Which stream does `mystery` represent?
  - The integers
  - The Fibonacci numbers
  - The factorials
  - The powers of 2

Check-In time!

Factor in some fun!

Look for Check-In I11A in the Source Academy!

Outline

Review

Streams as Recursive Definitions

Memoization

Applications

stream_ref

```
def enum_stream(low, hi):
    return None if low > hi else pair(low,
        lambda: enum_stream(low + 1, hi))
def stream_ref(s, n):
    return head(s) if n == 0 else stream_ref(
        stream_tail(s), n - 1)
s = enum_stream(1, 100)
```



- stream_ref(s, 0) $\Rightarrow$ 1
- stream_ref(s, 10) $\Rightarrow$ 11
- stream_ref(s, 99) $\Rightarrow$ 100

Recomputing Tails

- Every time we walk past an element of a stream, its tail function runs
- Two calls of `stream_ref(s, 3)` run the same three tail functions twice
- Idea: use *memoization* to remember the result of the first call

Without Memoization

- `ms(m, s)` prints `m` and returns `s`
- We put it into each tail to see when the tail runs



```
def ms(m, s):
    print(m)
    return s
def loud_from(n):
    return pair(n, lambda: ms("computing " + str(n + 1),
                              loud_from(n + 1)))
loud = loud_from(1)
stream_ref(loud, 2)
stream_ref(loud, 2)
```

- The first call prints `computing 2` and `computing 3`
- The second call prints them again: every tail runs again

Memoization: `memo_fun`

- `memo_fun(fun)` returns a function `mfun`
  - The first call of `mfun` runs `fun` and stores the result
  - Later calls return the stored result without running `fun`



```
def memo_fun(fun):
    already_run = False
    result = None
    def mfun():
        nonlocal already_run, result
        if already_run:
            return result
        else:
            result = fun()
            already_run = True
            return result
    return mfun
```

With Memoization

- Each tail is wrapped in `memo_fun`



```
def ms(m, s):
    print(m)
    return s
def memo_from(n):
    return pair(n, memo_fun(lambda: ms("computing " + str(n +
    1),
                                                memo_from(n + 1))))
memo = memo_from(1)
stream_ref(memo, 2)
stream_ref(memo, 2)
```

- The first call prints `computing 2` and `computing 3`
- The second call prints nothing: both tails are memoized

Why Don't We Always Memoize?

- Compare `loud_from` and `memo_from`
  - Without memoization, the `print` in the tail runs every time
  - With memoization, it runs only once
- Side effects
  - A function often does more than compute its value, for example printing
  - Memoization changes how often these side effects happen
- Pure functions have no side effects
  - They return the same value every time, so memoization never changes the result

Outline

Review

Streams as Recursive Definitions

Memoization

Applications

Example — Generating Primes

```
def is_prime(n):
    def check(d):
        return d * d > n or (n % d != 0 and check(d + 1))
    return n > 1 and check(2)
primes = pair(2, lambda: stream_filter(is_prime,
                                       integers_from(3)))
eval_stream(primes, 10)
```



```
[2, [3, [5, [7, [11, [13, [17, [19, [23, [29,
None]]]]]]]]]]
```

Can we make a prime stream without using `is_prime`?

Sieving for Primes: Multiples of 2

- Start with integers beginning with 2

  2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 ...

- Filter away multiples of 2 that are greater than 2

  2 3 5 7 9 11 13 15 17 ...

Sieving for Primes: Multiples of 3

- Next left over integer is 3

  2 3 5 7 9 11 13 15 17 ...

- Filter away multiples of 3 that are greater than 3

  2 3 5 7 11 13 17 ...

Sieving for Primes: Multiples of 5

- Next left over integer is 5

  2 3 5 7 11 13 17 ...

- And so on...

Sieving for Primes

```
def is_divisible(x, y):
    return x % y == 0

def sieve(s):
    return pair(head(s),
        lambda: sieve(stream_filter(
            lambda x: not is_divisible(x, head(s)),
            stream_tail(s))))
primes = sieve(integers_from(2))
eval_stream(primes, 10)
```



```
[2, [3, [5, [7, [11, [13, [17, [19, [23, [29,
None]]]]]]]]]]
```

Is This the Sieve of Eratosthenes?

- Not actually the sieve of Eratosthenes
  - The sieve crosses off multiples of each prime
  - `sieve` tests each candidate for divisibility by the primes found so far: *trial division*, substantially less efficient (O'Neill, *The Genuine Sieve of Eratosthenes*, 2009)

Square Roots by Newton's Method



```
def average(a, b):
    return (a + b) / 2
def good_enough(guess, x):
    return abs(guess * guess - x) < 0.001
def improve(guess, x):
    return average(guess, x / guess)
def sqrt_iter(guess, x):
    if good_enough(guess, x):
        return guess
    else:
        return sqrt_iter(improve(guess, x), x)
def sqrt(x):
    return sqrt_iter(1.0, x)
```

Using Streams for Iteration

```
def sqrt_stream(x):
    guesses = pair(1.0, lambda: stream_map(
        lambda guess: improve(guess, x), guesses))
    return guesses
eval_stream(sqrt_stream(2), 6)
```



```
[1.0, [1.5, [1.4166666666666665, [1.4142156862745097,
[1.4142135623746899, [1.4142135623373095, None]]]]]]
```

Summary

- In streams, functions serve as pickles
  - Open the jar (apply the function) only when you need them
- Streams can represent "infinite" data structures
- Laziness of streams avoids problem of non-termination
- Memoization can be used on streams
  - Each tail function runs at most once
  - It changes behavior only if the tail has side effects
- Streams can be used for iteration: `sieve`, `sqrt_stream`